

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

DIJKSTRA'S ALGORITHM, GRAPH TRANSFORMATIONS, AND A*



DIJKSTRA'S ALGORITHM

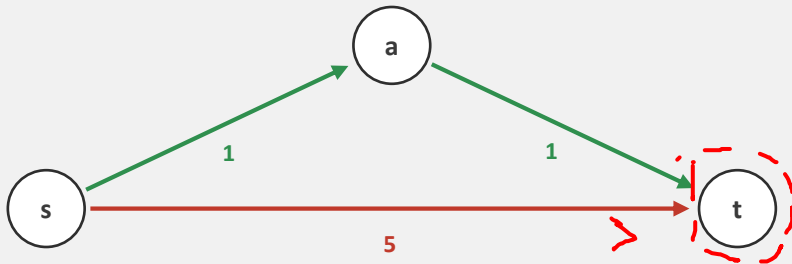


WEIGHTED GRAPHS AND WHY LAYERS FAIL

A weight function w assigns a number to every edge. The weight of a walk is the sum of its edge weights, and the question changes from fewest edges to least total weight.

Definition.

A weighted graph is a pair (G, w) where $G = (V, E)$ and w assigns a real number $w(u, v)$ to each edge. For a walk $v_1, v_2, \dots, v_k$ the weight is the sum of $w(v_i, v_{i+1})$ over $i = 1$ to $k-1$. Write $\text{dist}(s, v)$ for the smallest weight over all walks from s to v .



Breadth first search returns s, t, one edge, weight 5. The least weight walk is s, a, t: two edges, weight 2. Counting edges and adding weights are different questions the moment the weights differ.

SUBDIVIDING IS NOT A FIX

Replacing an edge of weight 5 by five edges of weight 1 is correct and unusable: an edge of weight 900 becomes 900 edges, so the running time depends on the numeric size of the weights rather than on n and m .

WHAT SURVIVES FROM BREADTH FIRST SEARCH

The skeleton survives: a container of frontier vertices, a rule for taking the next one out, a distance label, and a parent pointer. Only the removal rule changes, to smallest label first.

ASSUMPTION FOR THIS SECTION

Every weight is non negative, everywhere in this lecture. Slide 7 is where that is obvious



RELAXATION AND THE SETTLED SET

Every shortest walk algorithm in this course is built from one operation. Relaxation lowers a label when a lighter route through a neighbor is found.

```
def relax(u, v, w, d, parent):  
    if d[u] + w < d[v]:  
        d[v] = d[u] + w  
        parent[v] = u
```

WHAT THE LABELS MEAN AT EVERY MOMENT

OBJECT	INVARIANT IT SATISFIES
$d[v]$	The weight of the lightest walk from s to v found so far. Never below $\text{dist}(s, v)$, and only ever decreasing.
$\text{parent}[v]$	The vertex whose relaxation last lowered $d[v]$. Following parents from v back to s reconstructs a walk of weight exactly $d[v]$.
settled	The set of vertices for which $d[v]$ is already equal to $\text{dist}(s, v)$ and will not change again.

THE GREEDY STEP

Among all unsettled vertices, take one with the smallest label. Claim: that label already equals $\text{dist}(s, v)$, so it can be settled immediately.

WHY THAT CLAIM HOLDS

A competing walk must leave the settled set at some unsettled z . Its prefix weighs at least $d[z]$, and $d[z]$ is at least $d[v]$ since v had the smallest label. The tail adds at least zero.

RELAXATION IS THE SHARED PRIMITIVE

It is the only update in this lecture. Dijkstra schedules it by smallest label. A star, at the end, schedules it by smallest label plus an estimate of what remains. One operation, two schedules.



DIJKSTRA IN PYTHON

The frontier becomes a binary heap keyed by the label. Everything else is the traversal skeleton from Lecture 11.

```
import heapq

def dijkstra(adj, s):          # adj[u] = [(v, w), ...], w >= 0
    d = {u: float('inf') for u in adj}
    parent = {u: None for u in adj}
    d[s] = 0
    settled = set()
    pq = [(0, s)]

    while pq:
        -> du, u = heapq.heappop(pq)
        if u in settled:      # stale entry, already handled
            continue
        settled.add(u)
        for v, w in adj[u]:
            if du + w < d[v]:
                d[v] = du + w
                parent[v] = u
            heapq.heappush(pq, (d[v], v))
    return d, parent
```

Handwritten notes: "relax" with an arrow pointing to the relaxation loop; a bracket around the relaxation loop; a checkmark next to the push operation.

LAZY DELETION

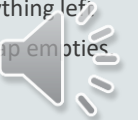
heapq has no decrease key, so a lower label is pushed as a second entry and the stale one is left in place. The settled test discards it when it surfaces. The heap holds up to m entries.

POPPING IS SETTLING

The first time a vertex leaves the heap its label equals $\text{dist}(s, v)$. Later entries are stale, since labels only decrease and the smaller value surfaced first.

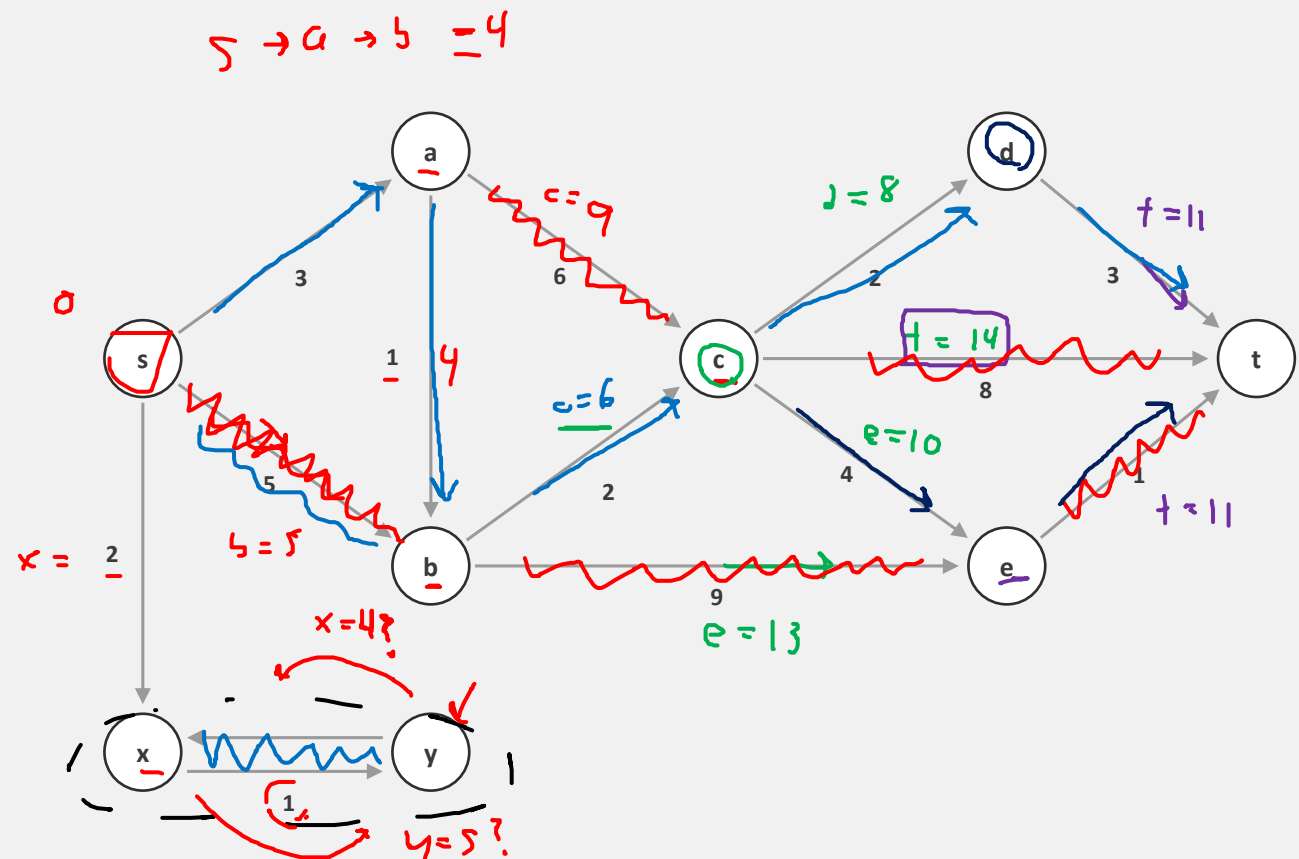
STOPPING EARLY

For one destination t , return the moment t is popped: everything left has a label at least $d[t]$. For all destinations, run until the heap empties.



A TRACE ON THE ROAD NETWORK

Directed edges, weights are travel times. Source is s. One row per pop, in the order the heap yields them.



Final labels: s 0, x 2, a 3, y 3, b 4, c 6, d 8, e 10, t 11. Two different walks from s to t weigh 11: s, a, b, c, d, t and s, a, b, c, e, t. The parent array records only one of them.

POPPED	LABEL	RELAXATIONS THIS POP
s	0	a to 3, b to 5, x to 2
x	2	y to 3
a	3	c to 9, b down to 4
y	3	none, x already settled
b	4	c down to 6, e to 13
c	6	d to 8, t to 14, e down to 10
d	8	t down to 11
e	10	none, t already at 11
t	11	none

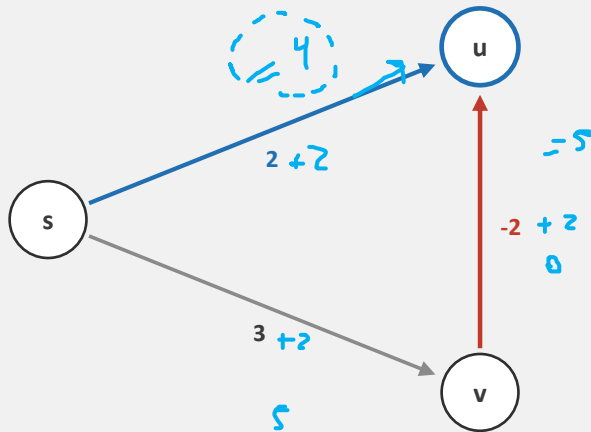


WHERE NON-NEGATIVITY IS USED

One sentence of the correctness argument needs $w \geq 0$. Remove it and the greedy step settles a vertex too early.

The sentence in question.

When v is settled, any competing walk leaves the settled set at some unsettled z , and the tail of that walk from z onward "adds at least zero." With a negative edge available, the tail can subtract, so a walk that looks heavy at z can finish lighter than $d[v]$.



Dijkstra settles u at 2, because 2 is the smallest label after s is popped. The true value is $\text{dist}(s, u) = 3 + (-2) = 1$, along s, v, u . By the time v is popped, u is closed and never reconsidered.

ADDING A CONSTANT DOES NOT REPAIR IT

Shifting every weight up by the same amount changes a walk of k edges by k times that amount, so it penalizes long walks and the minimizer moves. Slide 17 shifts by a function of the endpoints instead.

THE TEST TO APPLY

What breaks the argument is a negative edge pointing into a vertex that has already been settled, and there is no way to know in advance which vertex that will be. So the check is the plain one: scan the weights.

WHERE NEGATIVE WEIGHTS COME FROM

Weights modelling gain and loss rather than elapsed time: net elevation change along a route, a rebate applied to one leg of a shipment, or logarithms of exchange rates.



RUNNING TIME AND THE HEAP

Count heap operations, not vertices. Each successful relaxation pushes once, and every push is eventually popped.

STEP	HOW MANY TIMES	EACH ONE TAKES
Push a new label	at most m , one per successful relaxation	$\theta(\log m) = \theta(\log n)$
Pop the minimum	at most $m + 1$, everything pushed comes out	$\theta(\log m) = \theta(\log n)$
Settled membership test	once per pop	expected $\theta(1)$, it is a set
Scan $\text{adj}[u]$	once per vertex settled	$\theta(\deg(u))$, summing to $\theta(m)$

Total: $\theta((n + m) \log n)$ time, $\theta(n + m)$ space.

Since m is at most n^2 , $\log m$ is at most $2 \log n$, so the two logarithms are the same up to a constant and either form is acceptable on the exam. The n term inside the parentheses is there because every vertex is popped even in a graph with no edges.

THE ARRAY VERSION

Replace the heap by a linear scan for the smallest unsettled label. Each scan is $\theta(n)$ and there are n of them, and relaxation is $\theta(1)$, so the total is $\theta(n^2 + m)$.

WHICH ONE TO RUN

Dense graph, m near n^2 : the array version reads $\theta(n^2)$ against $\theta(n^2 \log n)$, so it wins. Sparse graph, m near n : the heap wins. The tradeoff is decided by m .

FIBONACCI HEAPS

Amortized $\theta(1)$ decrease key yields $\theta(m + n \log n)$. The best known bound, and almost never the fastest in practice.

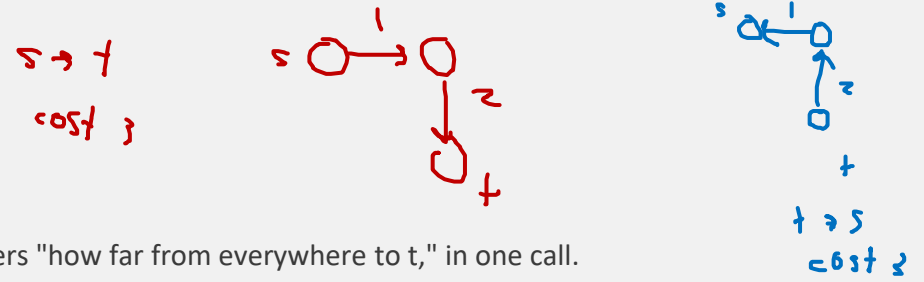


GRAPH TRANSFORMATIONS



THE REVERSE GRAPH

Dijkstra from s answers "how far from s to everywhere." Run it on the reverse graph from t and it answers "how far from everywhere to t ," in one call.



```
def reverse(adj):  
    # adj[u] = [(v, w), ...]  
    rev = {u: [] for u in adj}  
    for u in adj:  
        for v, w in adj[u]:  
            rev[v].append((u, w))  # flip the edge, keep w  
    return rev
```

```
rev = reverse(adj)  
to_t, _ = dijkstra(rev, t)  # to_t[v] == dist(v, t) in G
```

WHAT IT IS

The reverse of $G = (V, E)$ has the same vertices and the edge set $\{(v, u) : (u, v) \in E\}$, with weights carried along unchanged. Reversing twice returns G .

BUILDING IT

One pass over every adjacency list, so $\Theta(n + m)$ time and $\Theta(n + m)$ additional space. On an undirected graph the reverse is the graph itself, so this whole idea is a directed graph tool.

WHY A WALK IN THE REVERSE IS A WALK IN G

$v_1, v_2, \dots, v_k$ is a walk in the reverse graph exactly when $v_k, \dots, v_2, v_1$ is a walk in G , and the two have the same weight because each edge keeps its weight. So minimizing over one set minimizes over the other.

QUANTITY

HOW TO OBTAIN IT

$\text{dist}(s, v)$ for all v

Dijkstra on G from s

$\text{dist}(v, t)$ for all v

Dijkstra on the reverse of G from t

$\text{dist}(s, t)$ for one pair

Either of the above, read one entry

$\text{dist}(u, v)$ for all pairs

n runs of Dijkstra, or a different algorithm entirely

LAYERED AND PRODUCT CONSTRUCTIONS

When the rule about a route is not expressible in edge weights, put the missing information into the vertex. Then run an algorithm you already have.

01 Name the extra state, and make it finite.

Whatever the constraint remembers. Toll roads taken so far, capped at one. Fuel remaining, in whole units. Number of transfers used. Parity of the number of edges. Call that finite set S .

02 Build the product graph on $V \times S$.

A vertex is a pair (v, q) : you are standing at v and the extra state is q . An edge (u, q) to (v, q_2) exists when (u, v) is an edge of G and traversing it moves the state from q to q_2 . Weights are unchanged.

03 Run the unchanged algorithm, then read one entry.

Start at (s, q_0) and read the label of (t, q_{final}) , or the smallest label over all acceptable final states. The product has $n|S|$ vertices and at most $m|S|$ edges, so Dijkstra on it runs in $\Theta((n|S| + m|S|) \log(n|S|))$.

AT MOST ONE TOLL ROAD

Take $S = \{0, 1\}$, recording whether the toll edge has been used. Free edges keep the coordinate; a toll edge moves 0 to 1 and does not appear inside layer 1.

LAYERED GRAPHS FOR SCHEDULES

A network whose edges are available only at certain times becomes a graph on station and time slot pairs, with every edge pointing forward in time. No route can loop, because time never decreases.

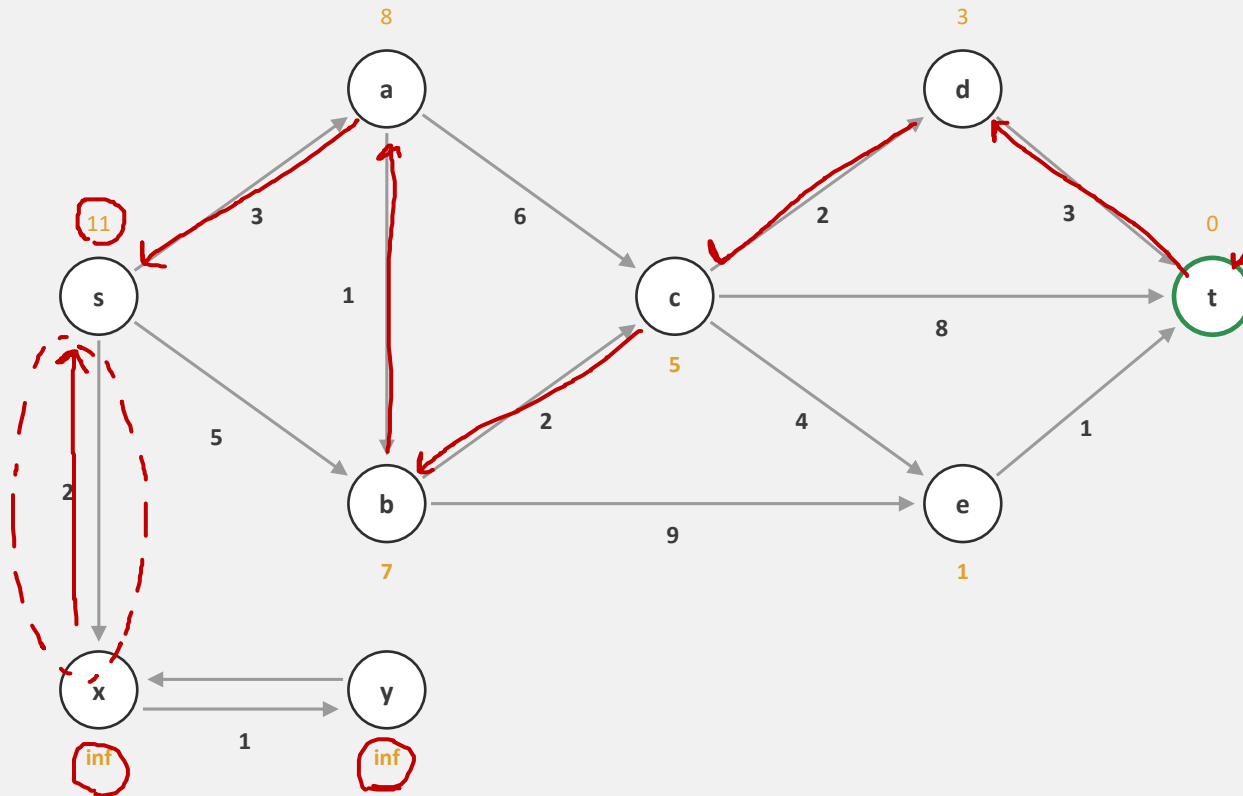
THE TRADEOFF BEING MADE

Nothing about the algorithm changes. The running time is multiplied by $|S|$. Worthwhile while $|S|$ is small, hopeless when the state is the set of vertices already visited.



PROBLEM: AN INTERACTIVE NAVIGATOR

The user enters s and t , then walks the network one vertex at a time. At every vertex the app lists the moves, recommends the ones still on a least weight walk to t , and flags the dead ends. Dijkstra may not be re-run once inside the loop.



The orange number under each vertex is $\text{dist}(v, t)$, not $\text{dist}(s, v)$. It is what the app has to report as remaining travel time, and it is produced by one Dijkstra run on the reverse graph from t .

01 Reverse the graph, once.

$\Theta(n + m)$ time and space, before the user moves at all.

02 Run Dijkstra on the reverse graph from t , once.

Label $\text{to_t}[v]$ holds $\text{dist}(v, t)$ in the original network. Vertices that cannot reach t keep the label infinity.

03 Keep the label array. Discard the heap.

Inside the interactive loop nothing is recomputed. Every answer is arithmetic on labels the app already holds.

WHY NOT DIJKSTRA FROM s

Labels out of s report distance already travelled. The app has to report distance still remaining, and it has to answer that from wherever the user happens to be standing, not from s . Only the reverse run produces that for every vertex at once.

A^*

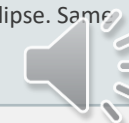


A graph diagram with nodes s, a, b, v, x, y . Node s is connected to a and b . Node a is connected to b . Node b is connected to v . Node v is connected to x and y . Node x is connected to y . A dashed blue oval encloses nodes x and y . A red box with a plus sign is to the right.



↓

Dijkstra settles everything within $\text{dist}(s, t)$ of the source, a disc. A^* delineates what has estimated total below $\text{dist}(s, t)$, an ellipse. Same answer but with fewer pops.



ADMISSIBILITY AND CONSISTENCY

Two conditions on the estimate h . Admissibility is a statement about one vertex at a time. Consistency is a statement about one edge at a time, and it is the one that turns A star into an ordinary Dijkstra run.

CONDITION	WHAT IT REQUIRES OF h	WHAT IT YIELDS
Admissible	$0 \leq h(v) \leq \text{dist}(v, t)$, at every vertex v	Correct, but only if a settled vertex may be reopened.
Consistent	$h(t) = 0$, and $h(u) \leq w(u, v) + h(v)$ on every edge	Correct with no reopening, at Dijkstra running time.

CONSISTENCY IS NON NEGATIVITY, AFTER ONE REWEIGHTING

$$w'(u, v) = w(u, v) - h(u) + h(v) \quad \text{the shifted weight}$$

consistency $h(u) \leq w(u, v) + h(v)$ says exactly $w'(u, v) \geq 0$

along a walk $p = v_1, v_2, \dots, v_k = q$:

$w(v_1, v_2) - h(v_1) + h(v_2)$
 $+ w(v_2, v_3) - h(v_2) + h(v_3)$
 $+ \dots$
 $+ w(v_{k-1}, v_k) - h(v_{k-1}) + h(v_k)$

$\xrightarrow{\quad}$

$w'(\text{walk}) = w(\text{walk}) - h(p) + h(q)$

every interior $h(v_i)$ appears twice, once with each sign

On a road map, $h(v)$ is the straight line distance from v to t divided by the highest speed anywhere on the network. No route beats that, so h is admissible, and euclidean distance obeys the triangle inequality, so h is consistent.

WHAT THE LAST LINE YIELDS

Fix the endpoints. Every walk from s to t is shifted by the same $h(t) - h(s)$, so w' and w rank those walks identically and select the same lightest one.

THE HEAP KEY IS THAT SAME SHIFT

Dijkstra on w' labels v with $g(v) - h(s) + h(v)$. A star's key is $g(v) + h(v)$. The two differ by $h(s)$, one constant for every vertex, so the two pop orders are identical.

WHEN ONLY ADMISSIBILITY HOLDS

Consistency implies admissibility, not the reverse. With an admissible but inconsistent h , w' is negative on some edge, the settled once argument fails for the reason slide 7 states, and a settled vertex can need reopening.



WHERE THESE RUN

Every deployed routing system is this lecture plus preprocessing. The algorithm is fixed; what varies is how much work is moved before the query arrives.

SYSTEM	GRAPH	WHAT IT ACTUALLY RUNS
Turn by turn navigation <i>Maps</i>	Road segments, weights are predicted travel times	A* with a geometric heuristic, on a network reversed from the destination when remaining time is wanted
Freight and shipping	Depots and lanes, weights are transit hours plus handling	Dijkstra on a layered graph whose extra state records vehicle hours already used
Transit planners	Station and departure time pairs	Dijkstra on the layered graph, where every edge points forward in time so no route can loop
Game and robot path finding <i>Pacman</i>	<u>Grid</u> cells or a navigation mesh	A* with an octile or euclidean heuristic, recomputed as obstacles move

PREPROCESSING IS THE REAL SPEEDUP

A continental road network has hundreds of millions of edges. Production engines precompute shortcut structures once, then answer a query in near constant work.

BIDIRECTIONAL SEARCH

Run one search forward from s and one backward from t on the reverse graph, stopping when they meet. Both frontiers grow at half the radius, a large constant factor on a planar network.

LANDMARK HEURISTICS

Store true distances from every vertex to a few dozen landmarks. The triangle inequality then yields an admissible, consistent heuristic far sharper than a straight line estimate.

